Lezione 18 - MultiThreading

Task e MultiTasking

Un processo è un programma in esecuzione (o una sua parte).
Quando avviamo un'applicazione, il sistema operativo crea un processo che dispone delle risorse necessarie per essere eseguito: memoria, tempo di CPU, file aperti, ecc.

Il **multitasking** è:

- **la capacità del sistema operativo di eseguire più processi "contemporaneamente".**
  In realtà, soprattutto su una singola CPU, si tratta di una **simulazione del parallelismo**:
- **la CPU passa rapidamente da un processo all'altro, dando l'impressione che vengano eseguiti nello stesso momento.**
  Su sistemi multi-core, invece, **più processi possono realmente essere eseguiti in parallelo.**

Il componente responsabile di questa gestione è lo **scheduler** del sistema operativo.
Lo scheduler decide:

- **quale processo deve essere eseguito,**
- **per quanto tempo può utilizzare la CPU,**
- quando effettuare il **context switch:**
  - **cioè il passaggio da un processo a un altro.**

Il context switch:

- **comporta il salvataggio dello stato corrente di un processo (registri, contatore del programma, ecc.) e il ripristino dello stato del processo successivo.**

Caratteristiche del multitasking

- Ogni processo possiede un **proprio spazio di indirizzamento in memoria** (spazio indirizzato).
- Questo significa che i processi sono isolati tra loro:

    - uno non può accedere direttamente alla memoria dell'altro.
- La **comunicazione tra processi (IPC – Inter Process Communication)** è più complessa e viene gestita dal sistema operativo tramite meccanismi specifici (pipe, socket, memoria condivisa, ecc.).

Collegamento con i Thread

I **thread** possono essere visti come:

- **unità di esecuzione più leggere rispetto ai processi.**
  A differenza dei processi:
- **condividono lo stesso spazio di memoria,**
- **appartengono allo stesso processo,**
- **collaborano per svolgere parti diverse dello stesso compito.**

Possiamo quindi pensare ai thread non come processi completamente indipendenti, ma come **sotto-attività coordinate che lavorano insieme per raggiungere un risultato comune, condividendo risorse e dati.**

Questo è il passaggio concettuale fondamentale:

- **il multitasking riguarda più processi separati,**
- **il multithreading riguarda più flussi di esecuzione all'interno dello stesso processo.**

Thread

Un **thread** è:

- **un flusso di esecuzione all'interno di un processo.**
  Se nel paragrafo precedente abbiamo visto che un processo è un programma in esecuzione con un proprio spazio di memoria, ora possiamo precisare che un processo può contenere **uno o più flussi di esecuzione**.

Fino ad ora abbiamo lavorato principalmente con programmi **mono-thread:**

- **cioè applicazioni che eseguono un unico flusso: il *thread principale* (`main`).**
  In questo caso l'esecuzione è sequenziale:

- **un’istruzione dopo l’altra.**

Tuttavia, a partire dal thread principale è possibile generare **thread secondari**, che permettono di suddividere il lavoro in più attività eseguite in modo concorrente. In questo senso:

- **un processo può essere mono-thread (un solo flusso),**
- **oppure multi-thread (più flussi di esecuzione interni).**

Un processo multi-thread può essere visto come:

- **un insieme di flussi che collaborano tra loro per raggiungere un obiettivo comune.**
  I thread non sono completamente indipendenti come i processi:
- **condividono la stessa memoria,**
- **condividono le stesse risorse del processo,**
- **possono quindi comunicare in modo diretto e più semplice.**

Thread principale e thread secondari

Ogni processo ha **almeno un thread principale**, **che rappresenta il punto di ingresso del programma.**
Da questo possono essere creati altri thread per delegare specifiche attività (ad esempio operazioni lunghe, I/O, calcoli complessi).

Priorità e Scheduler

I thread possono essere ordinati secondo una **priorità**.
La priorità rappresenta:

- **un suggerimento al** sistema operativo **su quale thread dovrebbe avere maggiore probabilità di ottenere tempo di** CPU**.**
-

È importante però sottolineare che:

- **lo** scheduler **del sistema operativo decide effettivamente quale thread eseguire,**
- **il programmatore può organizzare i flussi e impostare priorità,**
- **ma non controlla direttamente l’ordine reale di esecuzione.**

In altre parole, la gestione concreta della CPU rimane sempre a carico del sistema operativo.

Multithreading

Il **multithreading** indica la possibilità di:

- **eseguire più thread contemporaneamente all'interno dello stesso processo.**
  In pratica, **piccole unità di calcolo del programma possono essere eseguite quasi simultaneamente, sfruttando meglio la CPU e riducendo i tempi di esecuzione complessivi**.

✓ **Vantaggi**

- **Maggiore velocità di calcolo**:
  - **dividendo il lavoro in più thread, alcune operazioni possono essere eseguite in parallelo.**
- **Efficienza**:
  - **le risorse della CPU vengono utilizzate meglio, soprattutto nei programmi che prevedono operazioni lunghe o I/O.**
    Contents

Le caratteristiche sono

- **I thread diversi condividono lo stesso spazio di memoria del processo a cui appartengono** (JAVA Heap). Questo rende la **comunicazione tra thread molto più semplice** rispetto alla comunicazione tra processi separati.
- Tuttavia, la condivisione di memoria introduce **problemi di sincronizzazione**:
  - **se più thread accedono contemporaneamente agli stessi oggetti, possono verificarsi condizioni di race o incoerenze dei dati.**

JVM e Schedulazione dei Thread

La Java Virtual Machine (JVM) gestisce i thread utilizzando il criterio chiamato **preemptive-priority scheduling**, che combina diversi fattori per stabilire quale thread eseguire in un dato momento:

- **Priorità del thread**: **ogni thread ha un valore di priorità compreso tra 1 e 10.**

- **Ordine di arrivo**: **l'istante di tempo in cui un thread diventa pronto all'esecuzione (cioè entra nello stato *Runnable*).**
- **Time slicing**: **intervalli di tempo limitati della CPU assegnati a ciascun thread.**

Come lavora lo scheduler della JVM

1. **Schedulazione dei thread Runnable**:
   - **tutti i thread pronti all'esecuzione vengono considerati e viene stabilito per ciascuno un intervallo di tempo della CPU (time slicing).**
2. **Selezione per priorità**:
   - **tra i thread pronti, quello con la priorità più alta viene mandato in esecuzione.**
3. **Context switch**:
   - lo scheduler può interrompere un thread in esecuzione e passare a un altro thread nelle seguenti condizioni:
     1. **Arriva un thread con priorità maggiore.**
     2. **Il thread corrente è stato interrotto o ha terminato il task.**
     3. **Lo slot di tempo assegnato al thread corrente si è esaurito.**
4. **Gestione dei thread con stessa priorità**: **se più thread hanno la stessa priorità, vengono eseguiti secondo l'ordine di arrivo, applicando la logica First Come, First Serve (FCFS).**

In sintesi:

ogni thread, anche quello principale, viene schedulato secondo priorità, ordine di arrivo e time slicing.
Questo sistema permette alla JVM di gestire efficacemente più flussi di esecuzione concorrenti, massimizzando l'uso della CPU pur mantenendo una gestione ordinata dei thread.

Scheduler

Lo scheduler è il componente del sistema (gestito principalmente dal sistema operativo, con il supporto della JVM) che decide quale thread deve utilizzare la CPU in un determinato momento.
In un programma multithread possono esistere più thread in stato `Runnable`,

cioè pronti per essere eseguiti.
Tuttavia, la CPU può eseguire un solo thread per core alla volta.

Lo scheduler quindi:

- **seleziona uno dei thread in stato** `Runnable`
- **gli assegna un intervallo di tempo di CPU (*time slice*)**
- **può interromperlo per assegnare la CPU ad un altro** thread

La decisione non dipende dalla logica del nostro codice, ma da criteri come:

- priorità del thread
- **ordine di arrivo**
- **politiche interne di scheduling**
- **disponibilità dei core**

> ⚠ **Lo scheduler non conosce la logica dei nostri metodi.**
> Per lui, le istruzioni sono solo operazioni da eseguire, non blocchi logici "atomici".
>
> Ed è proprio da questo comportamento che nascono i problemi di concorrenza.
>
> **Esempio concreto:**
> Prendiamo in analisi questo codice:
>
> ```
> if(balance >= 200){
>  balance = balance - 200;
> }
> ```
>
> Supponiamo che il Thread A stia eseguendo la condizione dell' `if` .
> Subito dopo il controllo, lo scheduler può effettuare uno switch.
>
> A questo punto entra in esecuzione il Thread B, che potrebbe:
>
> - **controllare a sua volta il saldo**
> - **oppure arrivare direttamente alla sottrazione**
>
> Lo scheduler non sa che:

```
if + sottrazione
```

Deve essere un unica operazione logica, poiché vede solo istruzioni macchina.
Per lui sono una serie di istruzioni in sequenza, ovvero le vede come:

- istruzione 1
- istruzione 2
- istruzione 3
  E può interrompere tra una e l'altra.

```
^scheduler
```

Context Switch

Il context switch è il passaggio effettivo dall'esecuzione di un thread a un altro.
Quando lo scheduler decide di cambiare thread, avviene un'operazione tecnica chiamata *context switch*.

Durante questo passaggio:

1. **viene salvato lo stato del thread corrente (registri, program counter, stack)**
2. **viene caricato lo stato del nuovo thread**
3. **la CPU riprende l'esecuzione dal punto in cui quel thread era stato sospeso**

Quindi:

- **Lo scheduler decide il cambio**
- **Il context switch è l'operazione tecnica che lo realizza**

Riprendiamo il codice:

```
if(balance >= 200){
    balance = balance - 200;
}
```

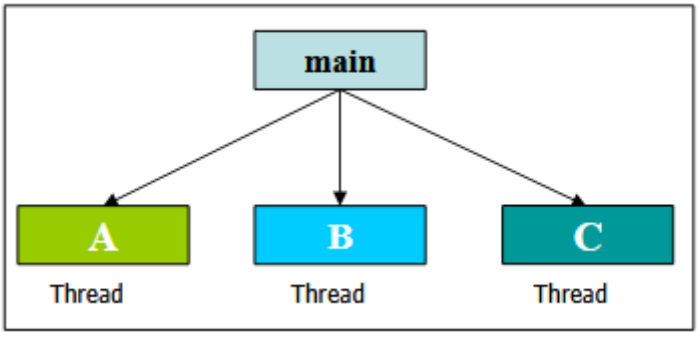
Può verificarsi questa sequenza:

- **Thread A controlla il saldo**
- **lo scheduler effettua un context switch**
- **Thread B modifica il saldo**
- **Thread A riprende ed esegue la sottrazione su un valore ormai cambiato**

> ⚠ **Nota importante:**
> Non è che "Thread A esegue solo l'if e Thread B solo la sottrazione".
> **È lo scheduler che può interrompere l'esecuzione tra le due istruzioni, creando l'inconsistenza.**
>
> Questo comportamento prende il nome di:
>
> **Race condition**

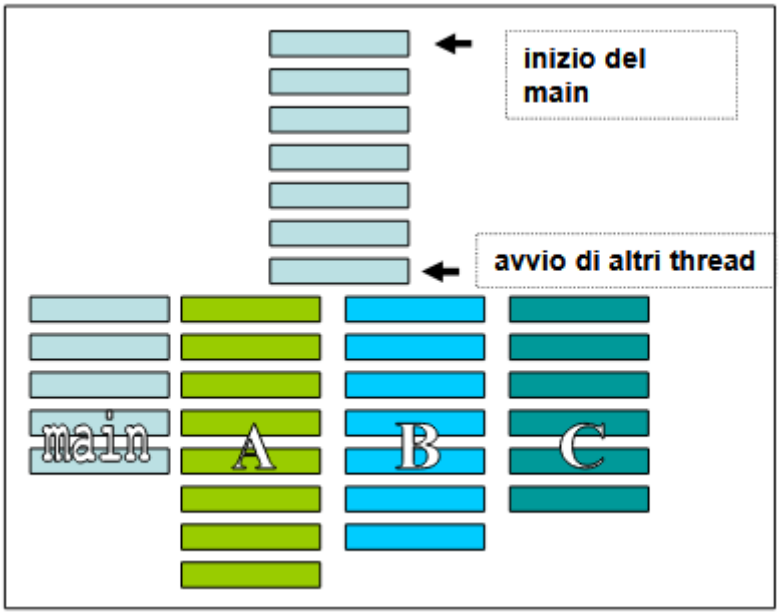
Il thread main

Il metodo `main` viene eseguito all'interno di un **thread principale**, chiamato anche **main thread**.



Da questo thread è possibile creare e avviare thread secondari, che lavorano in parallelo sia tra loro sia rispetto al thread principale.

Ogni thread, principale o secondario, **cerca di completare il proprio compito in modo indipendente, pur potendo interagire con gli altri thread quando necessario, ad esempio condividendo dati o risorse comuni.**



Classi di supporto nella libreria di Java

Il **multithreading** può essere implementato in molti linguaggi, ma Java fornisce delle API di base specifiche per gestirlo. Tutte si trovano nel package `java.lang` e le principali sono:

- **`interface Runnable`**
  - **Definisce il comportamento di un thread. Contiene un solo metodo astratto (`run()`), perciò è un'interfaccia funzionale che può essere implementata tramite lambda.**
- **`class Thread`**
  - **È una classe concreta che fornisce metodi per creare, avviare e gestire thread, come `start()`, `join()`, `sleep()`, e `interrupt()`.**
- `class Object`
  - Contiene metodi fondamentali per la **sincronizzazione**, come `wait()`, `notify()` e `notifyAll()`,
  - **utili per coordinare l'accesso concorrente alle risorse condivise e risolvere problemi di sincronizzazione tra thread.**

Creare un thread

Come abbiamo anticipato poco sopra, il comportamento di un thread è definito dall'**interfaccia `Runnable`**:

- **che rappresenta l'azione che il thread dovrà eseguire**:

```
public interface Runnable {
    // rappresenta il compito da eseguire dal thread
    public void run();
}
```

La **classe** `Thread` è:

- **un'implementazione concreta di** `Runnable` **e fornisce tutti gli strumenti necessari per creare e gestire thread.**
  Il metodo `run()` della classe `Thread` non esegue nulla di predefinito;
- **serve come punto di ingresso per il codice del thread.**

Per creare un thread in Java si possono usare due tecniche principali:

- **Estendere la classe** `Thread`**, sovrascrivendo il metodo** `run()`**.**
- **Implementare l'interfaccia** `Runnable`**, definendo il comportamento all'interno del metodo** `run()`**.**

> ⚠ **Nota:**
>
> **l'approccio con** `Runnable` **è utile quando la classe deve estendere un'altra classe, perché Java non consente l'ereditarietà multipla.**

Classe `Thread`

La classe `Thread`:

- **rappresenta un sotto-processo, ovvero un'unità di esecuzione indipendente all'interno di un processo.**
  Come abbiamo già anticipato poco sopra questa classe è l'implementazione concreta dell'interfaccia `Runnable`, e fornisce tutti gli strumenti necessari per **creare e gestire thread**

Costruttori principali:

- `Thread()` – **crea un thread senza specificare il nome o il target; il nome di default sarà** `"Thread-n"`**, dove** `n` **è un numero progressivo.**

- `Thread(Runnable target)` – **crea un thread associato a un oggetto** `Runnable`.
- `Thread(Runnable target, String name)` – **crea un thread con un target e un nome specifico.**
- `Thread(String name)` – **crea un thread con un nome specifico ma senza target.**

> **Nota:**
>
> **se non viene specificato il target** `Runnable`**, il thread non eseguirà nulla di predefinito**.
> Solo il thread principale (`main`) ha un nome di default `"main"`.

Metodi principali:

- `getName()` – **restituisce il nome del thread.**
- `setName(String name)` – **imposta il nome del thread.**
- `static currentThread()` – **ritorna il thread corrente in esecuzione.**

Questi costruttori e metodi permettono di creare, identificare e gestire thread in modo flessibile all'interno di un'applicazione multithread.

Utilizzo di Thread

Per creare e avviare un thread tramite estensione della classe `Thread` si seguono tre passi principali:

1. **Estendere la classe** `java.lang.Thread`**.**
2. **Ridefinire il metodo** `run()`**, che contiene il codice da eseguire nel thread.**
3. **Creare un'istanza del thread e avviarla tramite il metodo** `start()`**.**

> **Nota:**
>
> il metodo `start()`:
>
> - **inizializza e avvia il thread,**
> mentre `run()`:
> - **è il metodo che effettivamente esegue il codice del thread.**

🐞 **Non si dovrebbe mai chiamare `run()` direttamente, perché in tal caso il codice verrebbe eseguito nel thread corrente anziché in uno nuovo.**

Questo meccanismo, in cui `start()` prepara e invoca indirettamente `run()`, è un esempio di **Inversione di Controllo (IOC)**:

- **è il thread a gestire autonomamente l'esecuzione del metodo `run()`, non il programmatore.**

Schema

```
class MyThread extends Thread {
    @Override
    public void run() {
        // codice del thread
    }
}
```

Nel main;

```
public class Main {
    public static void main(String[] args) {
        MyThread tr = new MyThread();
        tr.start(); // avvia il thread
    }
}
```

Questo approccio consente di creare thread autonomi, ciascuno con il proprio flusso di esecuzione, pur rimanendo gestibile dal programmatore tramite l'oggetto `Thread`.

Avviare un Thread

Quindi, dopo aver creato un'istanza di `Thread` tramite il costruttore, **il thread non è ancora un flusso di esecuzione**.
Per trasformarlo in un thread operativo, bisogna chiamare il metodo `start()`.

Il metodo `Thread.start()` svolge tre compiti principali:

- **Crea le risorse di sistema necessarie per eseguire il thread.**
- **Schedula il thread per l'esecuzione.**
- **Chiama internamente il metodo `run()` della classe thread.**

> Dopo il ritorno di `start()`, **il thread entra nello stato Runnable e rimane pronto per essere eseguito secondo le regole dello scheduler.**

⚠ **Importante:**

- Non si dovrebbe mai invocare direttamente `run()`, perché in tal caso:
  - **il codice verrebbe eseguito nel thread corrente, non in un nuovo thread, e quindi non ci sarebbe multithreading.**
- **Gli oggetti che implementano `Runnable` da soli non si avviano, perché non hanno un metodo `start()`.**

Esempio pratico:

```
MyThread t = new MyThread();

// Esecuzione diretta del metodo run() -> NON multithreading
t.run();

for(int i = 0; i < 200; i++)
    System.out.println("ciclo - " + i);
```

In questo caso, il metodo `run()` **viene eseguito nel thread corrente, quindi prima termina `run()` e poi inizia il ciclo del `for`, senza esecuzione parallela.**

Per avere un vero multithreading, è necessario chiamare:

```
t.start(); // avvia il thread e il suo run() in parallelo
```

Così il thread `t` può lavorare contemporaneamente ad altri thread e al main thread.
Accade questo:

1. **La JVM crea le risorse di sistema per un nuovo thread.**

2. **Lo scheduler lo mette nello stato Runnable.**
3. **La JVM invoca automaticamente `run()` in un nuovo call stack, cioè in un nuovo flusso di esecuzione.**

Ed è qui che nasce il multithreading.

`run()` vs. `start()`

Per capire meglio:

1. `Runnable`

- È un'interfaccia funzionale che definisce **un solo metodo astratto**:

```
public void run();
```

Il metodo `run()` **contiene il comportamento che vogliamo far eseguire dal thread, ma non crea da solo un thread.**

1. `Thread`

- La classe `Thread` implementa `Runnable` (o può ricevere un `Runnable` come target).
- Ha un **metodo** `start()` che:
  - **Crea le risorse di sistema per il nuovo thread.**
  - **Schedula il thread per l'esecuzione.**
  - **Chiama internamente `run()` nel nuovo thread.**

Quindi la differenza chiave è:

- **Chiamare `run()` direttamente** → **viene eseguito nel thread corrente, nessun multithreading.**
- **Chiamare `start()`** → **il thread viene avviato, e Java invoca `run()` nel nuovo thread, parallelo al thread chiamante.**

In altre parole, **`start()` è il metodo che crea l'esecuzione parallelo, mentre `run()` contiene solo le istruzioni da eseguire.**

> **`start()` è un metodo concreto di Thread che implementa `run()`**

Utilizzo di Runnable

Quando non vogliamo (o non possiamo) estendere la classe `Thread`, possiamo definire il comportamento del thread implementando l'interfaccia `Runnable`.

Passaggi

Passo 1:

- **implementare l'interfaccia `Runnable` del package `java.lang`, effettuando l'override del metodo**:

```
public void run()
```

Il metodo `run()` contiene il codice che verrà eseguito dal thread.

Passo 2: **creare un oggetto della classe che implementa `Runnable`.**

Passo 3: creare un oggetto `Thread`, passando al costruttore l'oggetto `Runnable`:

```
Thread(Runnable target)
```

In questo modo il `Thread` riceve il comportamento da eseguire.

Passo 4: avviare il thread invocando `start()` sull'oggetto `Thread`.

Schema

```
class MyClass implements Runnable {

    public void run() {
        // codice eseguito dal thread
    }
}
```

Nel main;

```
MyClass p = new MyClass();
Thread tr = new Thread(p);
tr.start();
```

Questo è utile quando abbiamo una classe che non si estende alla classe `Thread` ma volgiamo che lavori come un thread.

🎟 Concetto chiave

- `Runnable` **definisce il comportamento**.
- `Thread` **gestisce l'esecuzione** (creazione, schedulazione, avvio).
- Il metodo `start()` inizializza il thread e chiama automaticamente `run()` in un nuovo flusso di esecuzione.

Questa soluzione è particolarmente utile quando:

- la classe deve già estendere un'altra classe (Java non supporta ereditarietà multipla),
- oppure si vuole separare in modo più pulito il *comportamento* (Runnable) dalla *gestione del thread* (Thread).

La priorità dei thread

Dopo aver visto come i thread vengono creati e schedulati, è importante capire un altro elemento che incide sul loro comportamento: **la priorità**.

Ogni thread in Java possiede una priorità numerica compresa tra **1 e 10**.
Questo valore rappresenta un *livello di preferenza* che lo scheduler può utilizzare per decidere quale thread mandare in esecuzione quando più thread sono nello stato Runnable.

Quando un thread viene creato, nasce con priorità **normale**, cioè pari a **5**. Questo valore corrisponde alla costante:

- `Thread.NORM_PRIORITY = 5`

La classe `java.lang.Thread` definisce anche:

- `Thread.MIN_PRIORITY = 1`
- `Thread.MAX_PRIORITY = 10`

Queste costanti delimitano l'intervallo teorico delle priorità.
Tuttavia, è importante sottolineare che **non è garantito che la JVM utilizzi realmente tutti e dieci i livelli in modo distinto**:

- **la gestione effettiva dipende anche dal sistema operativo sottostante.**

La priorità di un thread può essere:

- **letta tramite** `getPriority()`
- **modificata tramite** `setPriority(int)`

> ⓘ **Una precisazione importante:**
>
> **la JVM non modifica autonomamente la priorità di un thread** dopo la sua creazione:
>
> - **è il programmatore che può eventualmente intervenire.**

Detto questo, la priorità **non è una garanzia assoluta di esecuzione**.
Un thread con priorità più alta *non è matematicamente certo* che venga eseguito prima di uno con priorità più bassa. La priorità rappresenta solo un criterio preferenziale all'interno del meccanismo di schedulazione preemptive–priority scheduling che abbiamo visto in precedenza, insieme a:

- **ordine di arrivo**
- time slicing
- **possibili interruzioni**

In sintesi, la priorità influenza il comportamento dello scheduler, ma non determina in modo deterministico l'ordine reale di esecuzione dei thread.

User & daemon

I thread in Java si distinguono in **due categorie**:

1. **Thread user**
2. **Thread daemon**

Per impostazione predefinita, **tutti i thread nascono come user**.
È però possibile trasformarli in daemon tramite il metodo:

```
void setDaemon(boolean on)
```

Questo metodo **deve essere invocato prima della chiamata a `start()`**.
Se si passa `true` **:**

- **il thread viene impostato come daemon;**
  se si passa `false` **:**
- **rimane un thread user.**

Cosa significa essere un thread daemon?

Un **thread daemon** è un thread di servizio, generalmente utilizzato per attività di supporto in background (ad esempio monitoraggio, pulizia di risorse, gestione di servizi interni).

È subordinato ai thread user:

- **termina automaticamente quando tutti i thread user sono terminati.**

Questo è il punto fondamentale:

- **la JVM termina la propria esecuzione quando non esistono più thread user attivi, indipendentemente dal fatto che ci siano ancora thread daemon in esecuzione.**

In altre parole, la JVM "non aspetta" i daemon.

Relazione tra thread user e daemon

Dal punto di vista strutturale, **thread user e daemon sono identici**:

- **non cambia il modo in cui vengono creati, né la loro struttura interna.**

La differenza riguarda:

- **il loro ruolo (servizio vs lavoro principale)**
- **il comportamento in fase di chiusura della JVM**

Un thread daemon quindi non è un "thread più debole" dal punto di vista tecnico, **ma è vincolato alla vita dei thread user.**

Caso particolare: il thread main

Il `main` è un thread user.

Se dal `main` viene creato un thread daemon, ma esiste anche un altro thread user (diverso dal main), allora:

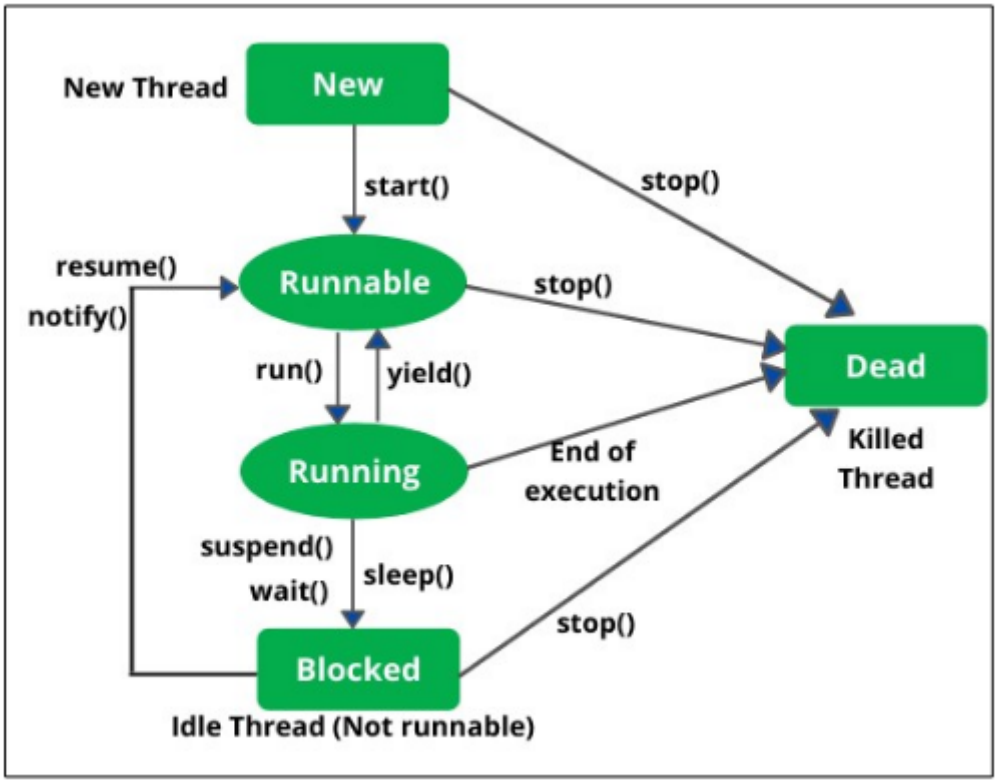
- **alla terminazione del `main`, il daemon non termina subito**
- **continuerà a vivere finché esiste almeno un altro thread user attivo**

La regola generale rimane:

> **la JVM termina solo quando tutti i thread user sono terminati.**

Ciclo di vita di un Thread

Per comprendere meglio come nasce, si evolve e termina un thread in Java, analizziamo il suo ciclo di vita.



Un thread, che viene definito dalla scheduler, attraversa diversi stati durante la sua esecuzione:

1. **New**:
   - **Il thread nasce quando viene creato con `new`.**
   - **In questo momento è solo un oggetto in memoria, non è ancora un thread di esecuzione.**
2. **Runnable**:
   - Quando si invoca `start()`, **il thread entra nello stato Runnable.**
   - Questo significa che è pronto per essere eseguito, ma non sta ancora utilizzando la CPU.
   - È lo scheduler della JVM che decide quando effettivamente mandarlo in esecuzione.
3. **Running**:
   - **Quando lo scheduler assegna la CPU al thread, questo passa in stato Running ed esegue il metodo `run()`.**

- **Un thread non rimane costantemente in Running: oscilla continuamente tra Runnable e Running in base alla gestione del processore.**

4. **Blocked (o Waiting)**:
   - **Un thread può essere temporaneamente sospeso.**
   - Questo avviene, ad esempio, se viene invocato `sleep()`, `join()` oppure `wait()`, oppure se deve attendere una risorsa condivisa.
   - **Solo un thread in esecuzione può entrare in questo stato.**
5. **Dead:**
   - **Quando il metodo `run()` termina, il thread entra nello stato Dead.**
   - Non significa che l'oggetto venga distrutto, **ma semplicemente che il suo ciclo di vita come thread è concluso e non può essere riavviato.**

Passaggi di Stato

Quindi, **un thread eseguibile oscilla continuamente tra Runnable e Running.**
Questo dipende dallo scheduler della JVM e dal meccanismo di time slicing:

- **il processore viene condiviso tra più thread.**

Un thread viene sospeso o bloccato quando:

- **vengono invocati `sleep()` o `join()`**
- **ci sono operazioni di I/O bloccanti**
- **viene chiamato `wait()` (per la sincronizzazione)**
- **viene invocato `suspend()` (metodo deprecato)**

È importante sottolineare che:

- **solo un thread in stato Running può passare a Waiting/Blocked. Quando esce da questo stato, torna in generale in Runnable, in attesa di essere nuovamente schedulato.**

Un thread termina (muore):

- **naturalmente, quando il metodo `run()` completa la sua esecuzione**
- **brutalmente, se viene chiamato `stop()` (metodo deprecato)**
- **accidentalmente, se si verifica un'eccezione non gestita**

ⓘ **I metodi `stop()` e `suspend()` sono deprecati perché possono generare stati inconsistenti;**

al loro posto si preferisce utilizzare il meccanismo di `interrupt()`.

Metodi bloccanti

Durante l'esecuzione di un thread (quindi quando si trova nello stato **Running**), è possibile forzarne temporaneamente la sospensione tramite alcuni metodi detti **bloccanti**.

Il primo è:

```
public static void sleep(long millis);
public static void sleep(long millis, int nanos);
```

Il **metodo `sleep()` è statico** e appartiene alla classe `Thread`.
Serve per sospendere il thread corrente per un intervallo di tempo specificato.

Cosa significa concretamente?

- **Il thread passa temporaneamente dallo stato Running allo stato Blocked/Waiting.**
- **Durante questo periodo non utilizza la CPU.**
- **Allo scadere del tempo torna nello stato Runnable, in attesa di essere nuovamente schedulato.**

⚠ **Importante: `sleep()` non libera eventuali lock posseduti dal thread. Sospende solo l'esecuzione.**

Un secondo metodo bloccante è `join()`.

```
public final void join();
public final void join(long milli);
public final void join(long milli, int nanos);
```

In pratica:

- **Il thread corrente si mette in attesa**

- **Attende la terminazione del thread su cui è stato invocato `join()`**

Esempio concettuale:
Se nel `main` si scrive:

```
t.join();
```

il thread `main` resta bloccato finché `t` non termina la sua esecuzione.

Anche in questo caso:

- **Il thread corrente passa in stato Blocked**
- **Quando il thread target termina (o scade il tempo massimo, se specificato), il thread corrente torna in Runnable**

Differenza concettuale tra `sleep()` e `join()`

- `sleep()` **sospende il thread per un tempo prefissato.**
- `join()` **sospende il thread finché un altro thread non termina.**

Nel primo caso il controllo è temporale, nel secondo è sincronizzato rispetto a un altro thread.

Entrambi sono strumenti fondamentali nella gestione del ciclo di vita dei thread e nel coordinamento tra flussi di esecuzione concorrenti.

Uso improprio di `sleep()`

Un errore molto comune riguarda l'uso del metodo `sleep()`.
Esempio:

```
public class Example {
    public static void main(String [] args) {
        Thread one = new Thread(); //istanzio l'oggetto Thread
        one.start(); //il thread entra nello stato di Runnable
        try {
            one.sleep(5*1000);
            // addormenta il thread corrente -> il main per 5
secondi
        } catch (InterruptedException ex) {
        }
```

```
    }
}
```

A prima vista potrebbe sembrare che `one.sleep(...)` metta in pausa il thread `one`.
In realtà **non è così**.

Il motivo è fondamentale:

- **`sleep()` è un metodo statico della classe `Thread`**
- **I metodi statici non agiscono sull'oggetto su cui vengono chiamati**
- **Agiscono sempre sul thread corrente**

Quindi in questo caso:

> **viene sospeso il thread main, non il thread `one`.**

Questo significa che:

- **Nessun thread può "addormentare" direttamente un altro thread.**
- **Ogni thread può sospendere solo sé stesso.**

Per questo motivo è buona pratica scrivere sempre:

```
Thread.sleep(5000);
```

così è chiaro che stiamo sospendendo il thread corrente.

Un concetto chiave da ricordare è che `sleep()`:

- **può essere invocato in qualsiasi punto del codice perché tutto il codice Java gira sempre dentro un thread (almeno il main).**

Esempio di `join()`

Vediamo ora un esempio pratico dell'uso corretto di `join()`.

```
public class ManyNames {
    public static void main(String [] args) {
        // istanzio un oggetto della classe NameRunnable
        NameRunnable nr = new NameRunnable();
        // istanzio tre oggetti di thread e l'inizializzo con
l'oggetto nr:NameRunnable
        Thread one = new Thread(nr);
```

```
        Thread two = new Thread(nr);
        Thread three = new Thread(nr);
        // avvio i Thread; dallo stato di new passano allo stato di
Runnable
        one.start();
        two.start();
        three.start();

        try {

            two.join();
            System.out.println("join con il TWO");
        } catch (InterruptedException e) {}
    }
}
```

Cosa accade qui?

- **Il `main` crea tre thread.**
- **Li avvia con `start()`.**
- **Subito dopo invoca `two.join()`.**

In questo momento:

- Il thread corrente (cioè il **main**) passa in stato **Blocked**.
- **Resta in attesa finché il thread `two` non termina.**
- **Solo dopo la fine di `two`, il `main` riprende l'esecuzione.**
- **Viene stampata la stringa `"join con il TWO"`.**

Quindi `join()` **crea una dipendenza temporale tra thread.**

> **Non impone un ordine di esecuzione tra `one`, `two` e `three`, ma impone che il `main` prosegua solo dopo la conclusione di `two`.**

Collegamento con il ciclo di vita

Nel ciclo di vita del thread:

- **Quando chiamiamo `join()` o `sleep()`**
- **Il thread corrente passa da Running a Blocked**
- **Quando termina l'attesa, torna in Runnable**
- **E successivamente potrà tornare in Running**

> Questo è il meccanismo fondamentale con cui coordiniamo l'esecuzione concorrente.

Da running a Runnable

In condizioni normali non abbiamo il controllo diretto sullo scheduler:

- **è la JVM, tramite il sistema operativo, a decidere quale thread eseguire e quando effettuare il *context switch*.**

Tuttavia esiste un metodo che consente, in modo molto limitato, di "interagire" con lo scheduler:

```
public static void yield()
```

Il metodo `yield()` è statico, come il metodo `sleep()`, e quindi agisce sul thread corrente.
Il suo scopo è;

- **quello di cedere volontariamente la CPU, suggerendo allo scheduler di effettuare uno *switch context* e di mandare in esecuzione un altro thread con priorità uguale o superiore rispetto a quella del thread attuale.**

È importante sottolineare un punto fondamentale:

> **`yield()` è solo una proposta, non un ordine.**

Dopo la chiamata a `yield()` può accadere che:

- **venga effettivamente schedulato un altro thread,**
- **oppure che venga nuovamente eseguito lo stesso thread che ha invocato `yield()`.**

!!!Non esiste alcuna garanzia.!!!

Dal punto di vista del ciclo di vita, il thread torna temporaneamente da **Running** a **Runnable**, rendendosi disponibile a lasciare la CPU.
Sarà poi lo scheduler a decidere se assegnarla ad un altro thread oppure no.

Per questo motivo `yield()` è poco utilizzato nella programmazione reale:

- **non offre controllo deterministico e il suo comportamento può variare a seconda della JVM e del sistema operativo sottostante.**

Interruzione

Abbiamo già anticipato sopra che i metodi `suspend()` e `stop()` sono stati dichiarati **deprecati** perché possono interrompere un thread in modo improvviso, **anche mentre sta eseguendo un'operazione atomica o si trova in una sezione critica.**
Questo comportamento può compromettere la coerenza dei dati e lasciare il programma in uno stato inconsistente.

A partire dalla versione 1.1 di Java sono stati quindi introdotti due metodi alternativi:

```
public void interrupt();
public static boolean interrupted();
```

Questi metodi implementano un meccanismo di interruzione meno "aggressivo".
Il thread non viene terminato forzatamente:

- **viene invece segnalata la richiesta di interruzione, lasciando al thread stesso la responsabilità di decidere quando e come terminare in modo sicuro.**

Funzionamento

- **Il thread che desidera interromperne un altro invoca il metodo `interrupt()` sull'oggetto che rappresenta il thread da interrompere.**
  - Questa chiamata **non termina il thread: ma imposta una flag interna che segnala la richiesta di interruzione.**
- **Il thread che può essere interrotto, dopo aver completato eventuali operazioni atomiche o sezioni critiche, verifica periodicamente la presenza della richiesta di interruzione tramite il metodo**:

```
Thread.interrupted();
```

- **Se il metodo restituisce `true`, significa che è stata richiesta l'interruzione.**

- **Dopo la chiamata, lo stato di interruzione viene automaticamente resettato (la flag viene riportata a `false`).**

> **NB:**
>
> **il metodo `interrupt()` non determina direttamente la chiusura del thread, ma si limita a impostare una flag.**
> **La terminazione effettiva dipende dalla logica implementata nel thread stesso.**

Esempio pratico

Supponiamo di avere due classi

- `Worker` → estende `Thread` e definisce il comportamento concorrente.
- `TestInterrupt` → contiene il `main` e avvia/interrompe il thread.
  Il punto chiave è l'interazione tra:

```
w.interrupt();          // richiesta di interruzione
Thread.interrupted();   // verifica della richiesta
```

```
public class Worker extends Thread {
@Override

public void run() {
    System.out.println("Thread avviato...");
    while (true) {
        for(int i = 0; i<=10; i++) {
            System.out.println(i);
        }
        System.out.println("Operazione completata!");
        // verifico della richiesta di interruzione
        if(Thread.interrupted()) {
            System.out.println("Interruzione rilevata. Chiusura del
thread in modo sicuro...");
            break;
            }
        }
        System.out.println("Thread terminato.");
    }
}
```

```java
public class TestInterrupt {
    public static void main(String[] args) {
        Worker w = new Worker();
        w.start();
        try {
            Thread.sleep(3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    System.out.println("Il main richiede l'interruzione del
Thread... ");
    w.interrupt();
    }
}
```

Analisi del `main`

```java
Worker w = new Worker();
w.start();
```

Qui:

- **Viene creato l'oggetto `Worker`**
- **`start()` crea un nuovo thread di esecuzione**
- **Il metodo `run()` viene eseguito in parallelo rispetto al `main`**

Poi:

```java
Thread.sleep(3000);
```

Il **thread main** si sospende per 3 secondi.

> ⚠ **Non si sta addormentando `Worker`, ma il thread corrente (cioè il main).**

> **Il metodo `sleep()` agisce SEMPRE sul metodo corrente (in questo caso il `main`)**

In seguito:
Dopo 3 secondi:

```
w.interrupt();
```

Questo NON interrompe immediatamente il thread.

Effetto reale:

- **Viene settata una flag interna di interruzione nel thread** `w`
- **Il thread continuerà a lavorare finché non controllerà quella flag**

Analisi del metodo `run()` **di Worker**
Al principio viene impostato un ciclo infinito.

```
while (true) {
```

Poi in seguito viene implementato un ciclo interno

```
for(int i = 0; i<=10; i++) {
    System.out.println(i);
}
```

Stampa i numeri da 0 a 10.

E fuori da questo ciclo viene stampato il messaggio di fine del blocco logico di lavoro

```
System.out.println("Operazione completata!");
```

In seguito viene impostato una condizione:

- è il controllo dell'interruzione

```
if(Thread.interrupted()) {
```

Cosa fa `Thread.interrupted()` **?**

- un metodo **statico**
- Controlla la flag di interruzione del **thread corrente**
- Se la flag è `true`:
  - **restituisce** `true`
  - **resetta la flag a false**

Quindi:

- **Prima che il main chiami `interrupt()` → la flag è `false`**
- **Dopo la chiamata → la flag diventa `true`**
- **Alla prima chiamata a `Thread.interrupted()` → ritorna `true` e la flag viene azzerata**

📋 Comportamento reale dell'esecuzione

Fase 1

Il thread parte e stampa ripetutamente:

```
0
1
...
10
Operazione completata!
```

Fase 2
Dopo 3 secondi il main stampa:

```
Il main richiede l'interruzione del Thread...
```

e chiama

```
w.interrupt();
```

Fase 3
Alla fine del ciclo corrente, quando il Worker esegue:

```
if(Thread.interrupted())
```

la flag è `true`.

Quindi:

```
Interruzione rilevata. Chiusura del thread in modo sicuro...
Thread terminato.
```

Il `break` fa uscire dal `while(true)` e il thread termina ordinatamente.

La sincronizzazione

Abbiamo accennato che i thread, condividendo lo **stesso spazio di memoria**, possono generare problemi di sincronizzazione.

Quando più thread collaborano su una stessa risorsa condivisa, è necessario garantire **coerenza dei dati**, altrimenti si verificano inconsistenze.

In altre parole:

- **due thread che lavorano per uno scopo comune devono coordinarsi nell'accesso alle risorse condivise.**

Esempio: il conto corrente (c/c)

La classe `Account` modella un semplice conto corrente condiviso tra più soggetti.

```
public class Account {

    private int balance = 1000;

    public int getBalance() {
        return balance;
    }

    public void withdraw(int amount) {
        if (balance >= amount) {
            System.out.println(Thread.currentThread().getName() + "
sta per eseguire il prelievo");

            balance = balance - amount;

            System.out.println("prelievo ok per " +
                    Thread.currentThread().getName() +
                    ", saldo attuale: " + balance);
        } else {
            System.out.println("ammontare non disponibile");
        }
    }
}
```

Osservazione importante

La classe `Account` non estende Thread.

Non rappresenta un'esecuzione concorrente, ma una **risorsa condivisa**.
Sono i thread (`AccountMng`) che agiscono su di essa.

Questo è concettualmente importante:

> **Il problema nasce quando più thread accedono alla stessa risorsa condivisa.**

Logica del metodo `withdraw`

Il metodo `withdraw` sembra corretto.
La sua logica è semplice e coerente:

1. Controlla se il saldo è sufficiente
2. Se lo è, esegue la sottrazione

In un programma sequenziale (senza thread) questo codice è perfettamente corretto.

Il problema nasce solo in presenza di concorrenza.

Problema concettuale

Il metodo `withdraw` contiene due operazioni distinte:

- **Controllo del saldo**
- **Aggiornamento del saldo**

Queste due operazioni, nel codice, sono consecutive.
Ma a livello di esecuzione reale **non sono atomiche**.

Tra il controllo e la sottrazione può intervenire un altro thread.

Ed è qui che nasce il problema.

Ipotizziamo uno scenario in cui sul conto ci sia un saldo iniziale di `200`, e abbiamo due thread (Fred e Lucy) che condividono lo stesso conto.

```
public class AccountMng extends Thread {

    private Account cc;
```

```java
    public AccountMng(String nome, Account cc) {
        super(nome);
        this.cc = cc;
    }

    // Il metodo run realizza 5 prelievi di 200
    public void run() {
        for (int x = 0; x < 5; x++) {
            cc.withdraw(200);
            System.out.println(">>>> Saldo attuale: " +
cc.getBalance());
        }
    }
}
```

Una sequenza possibile è:

```java
public class Simulazione {

    public static void main(String[] args) {

        // Creo il conto
        Account cc = new Account();

        // Creo i 2 thread con la stessa istanza del conto
        AccountMng fred = new AccountMng("Fred", cc);
        AccountMng lucy = new AccountMng("Lucy", cc);

        fred.start();
        lucy.start();
    }
}
```

- Fred entra in `withdraw(200)`
- Fred verifica `balance >= 200` → vero
- Lo scheduler sospende Fred
- Lucy entra in `withdraw(200)`
- Lucy verifica `balance >= 200` → vero
- Lucy sottrae → saldo = 0
- Lucy termina

- Fred riprende l’esecuzione
- Fred sottrae → saldo = -200 ❌

```
prelievo ok per Lucy, saldo attuale: -200
```

Questo accade perché il controllo e l'aggiornamento fatto nel metodo `withdraw()`:

```
if (balance >= amount)
balance = balance - amount;
```

non costituiscono un’operazione indivisibile.

Il controllo perde validità se, nel frattempo, un altro thread modifica lo stato del conto.

Questa situazione prende il nome di:

> **Race Condition**

Ovvero:

> **Il risultato finale dipende dall’ordine temporale imprevedibile di esecuzione dei thread.**

Quindi il codice non è logicamente sbagliato.

Il problema è strutturale:

> **Più thread accedono contemporaneamente a una sezione critica senza coordinarsi.**

La **sezione critica** è la parte di codice che accede o modifica una risorsa condivisa.

Nel nostro caso:

```
if (balance >= amount) {
    balance = balance - amount;
}
```

Questa porzione deve essere eseguita in modo esclusivo.

Qui ogni thread:

- esegue 5 prelievi da 200
- opera sulla **stessa istanza** di `Account`
  Questo verrà risolto nella versione sincronizzata (con `synchronized`)

Oggetto "occupato"

Abbiamo visto che il problema del conto corrente nasce perché lo **scheduler** decide autonomamente quando sospendere un thread ed eseguirne un altro.

Lo scheduler, però, **non conosce la logica del nostro metodo** `withdraw`.

Per lui, le istruzioni:

```
if (balance >= amount)
balance = balance - amount;
```

sono semplicemente due operazioni distinte.

Dal punto di vista logico, invece, esse dovrebbero costituire **un'unica operazione atomica**:

> controllo del saldo + aggiornamento del saldo devono essere inseparabili.

Il vero problema

Può accadere che:

1. **Un thread controlli il saldo**
2. **Lo scheduler effettui uno switch di contesto**
3. **Un altro thread modifichi il saldo**
4. **Il primo thread riprenda l'esecuzione basandosi su un controllo ormai non più valido**

Questo genera incoerenza del dato condiviso.

Il problema non è nella correttezza del codice, ma nel fatto che **l'oggetto** `Account` **è condiviso e accessibile contemporaneamente da più thread.**

La soluzione: `synchronized`

Per impedire che più thread eseguano contemporaneamente quella sezione critica, si utilizza la keyword:

```
synchronized
```

Applicandola al metodo `withdraw`, si impone una regola fondamentale:

> **Se un thread sta eseguendo quel metodo su un determinato oggetto, nessun altro thread può entrarci finché il primo non ha terminato.**

In altre parole:

- **L'oggetto `Account` viene bloccato (locked)**
- **Solo un thread alla volta può accedere al metodo sincronizzato**
- **Gli altri thread restano in attesa**

Cosa significa "oggetto occupato"?

Quando un metodo è dichiarato `synchronized`, **il thread che vi entra acquisisce il lock (monitor) dell'oggetto.**

Finché quel lock è detenuto:

- **L'oggetto è considerato "occupato"**
- **Gli altri thread che tentano di entrare nello stesso metodo vengono bloccati**

Il lock viene rilasciato automaticamente al termine del metodo.
La conseguenza quindi è che l'esecuzione non è più concorrente su quella sezione di codice.
I thread continuano a essere distinti e paralleli, ma:

> **Quella specifica parte di codice viene eseguita in modo mutuamente esclusivo.**

Quindi:

- **I thread non lavorano più contemporaneamente sul saldo**
- **Il controllo e l'aggiornamento diventano atomicamente protetti**
- **La consistenza del dato è garantita**

Example

Analogia del bagno occupato – comprendere `synchronized`

Immaginiamo che il **bagno** sia un oggetto condiviso, ad esempio il nostro `Account`.

Le **persone in fila** sono i thread (`Fred` e `Lucy`) che vogliono utilizzare quella risorsa.

Situazione senza sincronizzazione

Senza nessuna regola, può succedere questo:

- **Una persona entra.**
- **Sta ancora dentro.**
- **Un'altra persona entra comunque.**

Risultato?
Caos. Incoerenza. Violazione della regola di utilizzo esclusivo.

Nel caso del conto corrente succede la stessa cosa:

- **Thread A controlla il saldo.**
- **Prima che finisca l'operazione, entra Thread B.**
- **Entrambi modificano il saldo.**
- **Il conto va in rosso.**

Il problema è che **nessuno impedisce l'accesso contemporaneo alla risorsa condivisa**.

Situazione con oggetto "occupato" (`synchronized`)
Ora introduciamo una regola:

> Quando una persona entra nel bagno, chiude la porta a chiave.

Quella chiave rappresenta il lock dell'oggetto.

Finché la persona è dentro:

- **Il bagno è occupato**
- **Nessun altro può entrare**
- **Le altre persone devono aspettare fuori**

Quando la persona esce:

- Libera il bagno
- La chiave viene rilasciata

- Un’altra persona può entrare

La keyword `synchronized`

Un metodo dichiarato `synchronized`, quando viene invocato su un oggetto, **può essere eseguito da un solo thread alla volta su quell’oggetto.**

Modificando l’esempio precedente:

```
private synchronized void withdraw(int amount) {
    ...
```

si garantisce che le operazioni sul saldo siano **consistenti** e non soggette a condizioni di race.

Meccanismo di funzionamento

- Quando un thread entra in un metodo `synchronized`, **acquisisce il lock sull’oggetto corrente (`this`).**
- **L’oggetto risulta quindi bloccato, non il metodo.**
- Finché il lock è detenuto, **nessun altro thread può eseguire metodi `synchronized` sullo stesso oggetto.**
- Quando il thread esce dal metodo (normalmente o per eccezione), **rilascia automaticamente il lock.**
- **I thread eventualmente in attesa possono quindi tentare di acquisire il lock (l’ordine non è garantito in modo deterministico).**

Punto fondamentale

Il `synchronized`:

- **non fa parte della firma del metodo**
- **non impone vincoli sugli overriding**
- **riguarda il meccanismo di sincronizzazione, non il contratto del metodo**

Cosa viene sincronizzato realmente

È importante chiarire che:

- **Non viene bloccato il metodo.**
- **Viene bloccata l'istanza dell'oggetto su cui il metodo è invocato.**

Questo significa che:

- **Se due thread lavorano su oggetti diversi, non si bloccano a vicenda.**
- **Se lavorano sullo stesso oggetto, uno solo può eseguire codice `synchronized` alla volta.**

Applicazione all'esempio `Account`

Nel caso dell'oggetto `Account`, è il metodo `withdraw` che deve essere `synchronized`, **perché è lì che avviene la modifica dello stato condiviso (`balance`).**

Anche inserendo una `sleep` dentro il metodo, la correttezza resta garantita:

- **il lock rimane detenuto fino all'uscita dal metodo.**

Se invece sincronizzassimo il metodo `run()` in `AccountMng`, il primo thread che entra bloccherebbe l'intero ciclo di operazioni (ad esempio tutti e 5 i prelievi), impedendo l'alternanza tra i thread.
In quel caso la sincronizzazione sarebbe troppo ampia.

<> Buona pratica

Il `synchronized` va usato con parsimonia:

- **sincronizzare solo la sezione critica necessaria**
- **evitare di estendere inutilmente il blocco**
- **ridurre il tempo in cui il lock è detenuto**

Il lock e `synchronized`

Il concetto di lock

- **Ogni oggetto in Java possiede un solo lock (monitor).**

- Quando un thread entra in un metodo o blocco `synchronized`, **acquisisce il lock dell'oggetto.**
- Finché il lock è detenuto, **nessun altro thread può eseguire codice** `synchronized` **sullo stesso oggetto.**

Relazione tra thread e lock

- **Un thread può acquisire più lock, ma solo se appartengono a oggetti diversi.**
- Un thread in `sleep()` **non rilascia il lock**:
  - **se entra in** `sleep` **mentre è dentro una sezione** `synchronized`**, continua a detenere il lock fino all'uscita dal metodo o blocco sincronizzato.**
- **Il lock viene rilasciato automaticamente solo quando il thread esce dalla sezione sincronizzata.**

Cosa può essere sincronizzato

- Possono essere `synchronized`:
  - **metodi**
  - **blocchi di codice**
- Non possono essere sincronizzati:
  - **variabili**
  - **classi (nel senso diretto del termine)**

È fondamentale comprendere che:

> **Non si sincronizzano i thread, ma l'accesso concorrente dei thread agli oggetti.**

La sincronizzazione protegge lo stato condiviso, non l'esecuzione del thread in sé.

⚠ Attenzione all'uso eccessivo

Non è buona pratica rendere tutti i metodi `synchronized`.

Motivi:

- **La sincronizzazione introduce un vincolo di mutua esclusione.**
- **Riduce il parallelismo effettivo.**

- **Se progettata male può portare a deadlock (ad esempio quando due thread si bloccano a vicenda aspettando lock posseduti dall'altro).**

Principio generale

Sincronizzare:

- **solo le sezioni critiche**
- **solo gli accessi a stato condiviso**
- **per il tempo strettamente necessario**

L'obiettivo non è "bloccare tutto", ma garantire coerenza senza compromettere inutilmente la concorrenza.

A prova di Thread (Thread safe)

Una classe è **thread-safe** quando:

- **garantisce la coerenza del proprio stato anche in presenza di accessi concorrenti.**
  In particolare:
- Una classe è thread-safe se **tutti i metodi che accedono a proprietà modificabili (stato mutabile) sono implementati come** `synchronized`**, oppure sono protetti tramite meccanismi equivalenti di sincronizzazione.**
- **L'obiettivo è evitare inconsistenze dovute a modifiche concorrenti.**

Nel package delle collection esistono classi già thread-safe, ad esempio:

- `Vector` → **implementazione di** `List`
- `Hashtable` → **implementazione di** `Map`

Queste classi sincronizzano internamente i metodi di accesso.

Inoltre, la classe `Collections` mette a disposizione metodi wrapper per rendere sincronizzate collezioni non thread-safe:

- `Collections.synchronizedList(List list)`
- `Collections.synchronizedSet(Set set)`
- `Collections.synchronizedMap(Map map)`

Questi metodi restituiscono una vista sincronizzata della collezione.

> ⚠ **Importante:**
>
> rendere una classe thread-safe non significa semplicemente “mettere `synchronized` ovunque”, ma proteggere correttamente lo stato condiviso.

Se il lock non può essere rilasciato?

Supponiamo che:

- **Un thread entri in un metodo `synchronized`**
- **Acquisisca il lock dell’oggetto**
- **Ma non ci siano le condizioni logiche per eseguire il lavoro (esempio: saldo insufficiente per un prelievo)**

Cosa può fare?

Opzione 1

- **Esce dal metodo**
- **Rilascia il lock**
- **Eventualmente senza aver svolto il lavoro**

> ⚠ **Problema:**
>
> - Non sappiamo se l’operazione è stata completata o meno
> - Il comportamento può risultare incoerente rispetto alla logica applicativa

Opzione 2

- Rimane nel metodo
- Attende che qualcosa cambi
- NON rilascia il lock

> ⚠ **Problema:**
>
> - Finché mantiene il lock, nessun altro thread può intervenire per modificare lo stato

- La situazione potrebbe non evolvere mai

In questo caso si parla di **Thread Starvation**:

- **un thread rimane “affamato” di risorse per lungo tempo perché non riesce a progredire.**

La soluzione corretta

La soluzione è:

- **Rimanere nel metodo**
- **Non eseguire il lavoro finché la condizione non è soddisfatta**
- **MA rilasciare temporaneamente il lock**

Questo comportamento si ottiene tramite i metodi:

- `wait()`
- `notify()`
- `notifyAll()`

Questi metodi appartengono alla classe `Object`, quindi sono ereditati da tutti gli oggetti.

Cosa fa `wait()`

- **Può essere invocato solo dentro un contesto** `synchronized`
- **Fa rilasciare il lock al thread corrente**
- **Mette il thread in stato di attesa**
- **Il thread rimane sospeso finché non viene notificato**

Cosa fa `notify()`

- **Risveglia uno dei thread in attesa sullo stesso oggetto**
- **Il thread risvegliato non riparte immediatamente**
- **Prima deve riacquisire il lock**

Cosa fa `notifyAll()`

- **Risveglia tutti i thread in attesa su quell’oggetto**
- **Solo uno alla volta riuscirà a riacquisire il lock**

⚠ Nota importante

I metodi `wait()`, `notify()` e `notifyAll()`:

- **Sono definiti in `Object`**
- **Sono dichiarati `final`**
- **Non possono essere overridden**
- **Possono essere usati solo quando il thread possiede il lock dell'oggetto**

Questo perché il meccanismo di attesa e notifica è strettamente legato al monitor (lock) interno di ogni oggetto.

🎟 Concetto chiave

Con `wait()` e `notify()`:

- **Non si forza un thread ad uscire**
- **Non si blocca definitivamente l'oggetto**
- **Si coordina l'accesso concorrente in modo controllato**

L'idea è:

> **sospendere temporaneamente un thread senza impedire agli altri di modificare lo stato necessario al suo avanzamento.**

Waiting e Notifying

I metodi:

- `wait()`
- `notify()`
- `notifyAll()`

appartengono alla classe `Object` e possono essere invocati solo all'interno di un contesto `synchronized`.

Se un thread prova a chiamarli senza detenere il lock dell'oggetto, viene lanciata una:

```
IllegalMonitorStateException
```

Principio fondamentale

- **Solo un thread che detiene il lock su un oggetto può invocare `wait()`, `notify()` o `notifyAll()` su quell'oggetto.**
- **Il lock è associato all'oggetto.**
- **L'azione però si "scatena" sul thread.**

In altre parole: il monitor appartiene all'oggetto, ma è il thread che cambia stato.

`wait()`

Il metodo `wait()` viene chiamato dal thread che deve attendere una certa condizione.

Quando un thread:

1. **Entra in un metodo `synchronized`**
2. **Detiene il lock dell'oggetto**
3. **Incontra `wait()`**

succede quanto segue:

- **Il thread non esce dal metodo**
- **Il thread rilascia il lock**
- **Il thread entra nello stato di *waiting***
- **Diventa "trasparente" rispetto all'oggetto, cioè permette ad altri thread di entrare nel metodo sincronizzato**

Questo è il punto chiave:

> `wait()` **sospende il thread MA libera il lock.**

Così altri thread possono modificare lo stato dell'oggetto, rendendo eventualmente vera la condizione attesa.

Quando il thread viene risvegliato:

- **Deve prima riacquisire il lock**
- **Poi riprende l'esecuzione dal punto successivo alla `wait()`**

`notify()`

Il metodo `notify()` viene usato per:

- **risvegliare uno dei thread che si trovano in stato di waiting su quell'oggetto.**

Importante:

- **Il thread che chiama `notify()` deve detenere il lock.**
- **Il thread risvegliato non riparte immediatamente.**
- **Prima deve attendere che il lock venga rilasciato.**

Se `notify()` è invocato dentro un metodo sincronizzato:

- **Il metodo termina normalmente**
- **Solo alla fine rilascia il lock**
- **Solo allora uno dei thread notificati può riacquisirlo e proseguire**

Quindi:

> La notifica non implica rilascio immediato del lock.

`notifyAll()`

`notifyAll()` funziona come `notify()`, ma:

- **Risveglia tutti i thread in attesa su quell'oggetto.**
- **Solo uno alla volta riuscirà comunque a riacquisire il lock.**

Si utilizza quando:

- **Ci sono più thread in attesa**
- **Non si vuole rischiare di risvegliare quello "sbagliato"**

📋 Sequenza logica completa

1. Thread A entra in un metodo `synchronized`
2. Verifica una condizione
3. Se la condizione non è soddisfatta → chiama `wait()`
4. Rilascia il lock e va in attesa
5. Thread B entra nel metodo
6. Modifica lo stato dell'oggetto
7. Chiama `notify()` o `notifyAll()`

8. Termina il metodo e rilascia il lock
9. Thread A riacquisisce il lock e continua

⚠ Concetto chiave finale

- `wait()` → **sospende il thread e libera il lock**
- `notify()` → **segnala che qualcosa è cambiato**
- `notifyAll()` → **segnala a tutti i thread in attesa**
- **Il lock è sempre legato all'oggetto**
- **Il cambio di stato riguarda il thread**

Questo meccanismo consente una cooperazione controllata tra thread senza generare starvation o blocchi permanenti, se utilizzato correttamente.**

Overloading di `wait`

Quindi abbiamo detto che il metodo `wait` presente in tutte le istanze di `Object`, che **permette a un thread di sospendersi temporaneamente fino a ricevere una notifica.**

Il metodo `wait` è **overloaded**, ovvero esistono più versioni:

- `public final void wait()`
  Sospende il thread corrente indefinitamente fino a quando un altro thread invoca `notify()` **o** `notifyAll()` **sull'oggetto monitor.**
- `public final void wait(long timeout)`
  Sospende il thread per un massimo di `timeout` **millisecondi, oppure fino a ricevere una notifica.**
- `public final void wait(long timeout, int nanos)`
  Sospende il thread per un massimo di `timeout` **millisecondi più** `nanos` **nanosecondi, oppure fino a ricevere una notifica.**

Tutte queste versioni devono essere invocate all'interno di un blocco `synchronized`**, perché il thread deve possedere il monitor dell'oggetto su cui chiama** `wait`**.**
Questo garantisce che il rilascio temporaneo del monitor sia corretto e che non ci siano condizioni di race.

Gestione delle eccezioni

Il metodo `wait` può essere interrotto da un altro thread, esattamente come i metodi `sleep` e `join`.
Per questo motivo, tutte le versioni di `wait` dichiarano l'eccezione checked `InterruptedException`, che deve essere gestita o rilanciata:

```
synchronized(obj) {
    try {
        obj.wait(); // idem per wait(timeout) e wait(timeout, nanos)
    } catch (InterruptedException e) {
        // gestione dell'interruzione del thread
        e.printStackTrace();
    }
}
```

Se si preferisce, invece di gestirla localmente, è possibile dichiarare `throws InterruptedException` sul metodo chiamante, lasciando che l'eccezione venga propagata.

È importante sottolineare che il metodo `notify()` (o `notifyAll()`) non lancia eccezioni checked, quindi può essere chiamato direttamente all'interno di un blocco `synchronized` per risvegliare i thread in attesa senza necessità di try-catch.

<> Considerazioni pratiche

- `wait()` **blocca il thread fino a notifica.**
- `wait(timeout)` **blocca per un tempo massimo, utile per evitare blocchi indefiniti.**
- `wait(timeout, nanos)` **offre maggiore precisione temporale.**
- Sempre usare all'interno di `synchronized`.
- Gestire `InterruptedException` o rilanciarla.
- `notify()` / `notifyAll()` **risvegliano i thread senza eccezioni checked.**

Situazioni critiche nella programmazione concorrente

Lavorare con i thread in Java è sostanzialmente complesso e richiede attenzione, perché i thread possono **contendersi il lock** su un oggetto condiviso.

Questa contesa è nota come **Thread Contention** e **rappresenta una delle principali fonti di problemi nelle applicazioni multithread.**

Quando più thread competono per lo stesso monitor o risorsa, possono verificarsi situazioni critiche che richiedono gestione specifica. Tra le più comuni troviamo:

- **Thread Starvation**
- **Thread Deadlock**
- **Thread Live Lock**

Starvation

Il **Thread Starvation** si verifica quando:

- **due o più thread si contendono lo stesso lock, ma uno dei thread riesce a ottenere il lock in modo privilegiato e lo rilascia raramente.**

In questo scenario, gli altri thread restano **"affamati"**:

- **pur essendo pronti ad eseguire, rimangono in attesa indefinita del lock e non riescono mai a proseguire con il loro lavoro.**

Esempio concreto:

- **Thread A e Thread B devono accedere a una risorsa condivisa.**
- **Thread A ottiene il lock frequentemente e per tempi lunghi.**
- **Thread B resta bloccato in attesa, senza mai riuscire a eseguire le proprie operazioni.**

Questa situazione può degradare significativamente le prestazioni e portare a comportamenti imprevedibili se non gestita correttamente.

Deadlock

Il **Deadlock** si verifica quando:

- **due o più thread rimangono bloccati, ciascuno in attesa che l'altro rilasci un lock necessario per proseguire.**

Esempio semplificato:

- **Thread A possiede il lock sull'oggetto X e aspetta il lock sull'oggetto Y.**
- **Thread B possiede il lock sull'oggetto Y e aspetta il lock sull'oggetto X.**

In questo scenario, nessuno dei due thread può progredire, e il programma resta **bloccato**.

Strategie di prevenzione:

- **Progettare correttamente l'ordine di acquisizione dei lock.**
- **Ridurre al minimo le sezioni sincronizzate.**
- **Utilizzare strumenti di sincronizzazione avanzati come** `ReentrantLock` **con timeout.**

Live Lock

Il **Live Lock** è simile al Deadlock, ma con una differenza chiave:

- **i thread non sono bloccati in modo passivo, ma continuano ad eseguire operazioni senza mai completare il lavoro.**

Esempio pratico:

- **Due auto si trovano su un ponte a corsia unica e viaggiano in direzioni opposte**.
- **Ciascuna auto cerca di cedere il passo all'altra per agevolarla.**
- **Se entrambe adottano questa strategia contemporaneamente, vanno avanti e indietro senza mai riuscire a superare il ponte.**

In termini di thread, **ciò significa che i thread continuano a cedere o rilasciare lock nel tentativo di evitare conflitti, ma nessuno riesce a completare la propria operazione.**

Sintesi delle principali criticità

Situazione	Causa	Effetto
Starvation	**Un thread ottiene il lock frequentemente**	Thread affamati in attesa indefinita
Deadlock	**Thread in attesa circolare di lock**	Nessun thread progredisce, blocco totale
Live Lock	**Thread cedono il lock reciprocamente**	Nessun thread fa progressi pur eseguendo operazioni

Questi fenomeni mostrano quanto sia delicata la programmazione concorrente in Java:

- **la progettazione dei thread e l'uso corretto di `synchronized`, `wait` e `notify` sono fondamentali per evitare problemi di contesa e garantire l'esecuzione corretta del programma.**